Git, from zero

This assumes you have never typed a git command in your life. If you already know what a commit and a branch are, skim for the Gitea-specific parts and the merge-conflict walkthrough, then stop.

Read this **before Project 1 is due** — you cannot submit anything in this course without pushing it, and `git push` is meaningless if you don't know what a commit is yet.

Why any of this exists

Without version control, "my project" is whatever is currently on disk. If you break something, your only history is what you remember. Git keeps a complete, timestamped history of every version of every file you've told it to track, so you can always get back to a version that worked — and so two people (your P2 partner, your P3 team) can work on the same project without overwriting each other's changes.

Gitea, not GitHub. This course hosts repositories on a self-hosted Gitea instance, not GitHub. Gitea speaks the same `git` protocol and looks similar in the browser, so everything below works identically — only the web address is different. Your instructor gives you that address.

Four words you need before anything else makes sense

Word	What it actually means
Repository ("repo")	A project folder that git is tracking. Marked by a hidden `.git` folder inside it — never edit that folder by hand.
Working directory	The actual files on disk, as you see them in VS Code right now.
Staging area	A holding pen for changes you're about to save. You choose what goes in with `git add` before you commit.
Commit	A saved snapshot of the staging area, with a message explaining why. This is the unit of history — not "the file," the *commit*.

A repository's history is just a chain of commits, each one a snapshot plus a pointer to the commit before it.

One-time setup

Already done if you followed `docs/SETUP_WSL2.md` step 4. If not:

```
git config --global user.name  "Your Name"
git config --global user.email "your.email@university.edu"
git config --global init.defaultBranch main
```

The solo workflow: status → add → commit → log

Do this inside `cpp-course-starter/` (or whatever project you're in) after making some changes.

```
git status
```

Shows what changed since your last commit: modified files, new files git doesn't know about yet ("untracked"), and what's already staged. **Run this constantly.** It is the single most useful git command and it changes nothing — you cannot break anything by running `git status`.

```
git add src/grid.cpp
```

Stages that one file — puts it in the holding pen for the next commit. To stage everything that changed: `git add -A`. Run `git status` again and you'll see it move from "Changes not staged" to "Changes to be committed."

```
git commit -m "Fix off-by-one in Grid::row"
```

Takes a snapshot of everything currently staged, with that message attached. **Write messages that explain *why*, not what** — `git log` will show you what changed; it can't tell you why you thought it was a good idea.

```
git log --oneline
```

Shows the commit history, newest first. `git log` (no `--oneline`) shows full messages, author, and date for each one.

One commit per idea. Five small, clearly-labeled commits ("add board struct," "add flood-fill reveal," "fix reveal recursion base case") are far more useful — to you, to a partner, to a grader — than one giant "did the project" commit at 11:58pm.

Working with Gitea: clone, push, pull

Clone — get a copy of a repository that already exists on Gitea (you did this in `docs/SETUP_WSL2.md` step 5):

```
git clone https://git.pdulab.net/pdu/2440099 starter
```

This downloads the whole history, not just the latest snapshot, and remembers where it came from (called `origin`).

Push — send your local commits up to Gitea. This *is* the submission mechanism in this course — nothing you haven't pushed exists as far as grading is concerned:

```
git push
```

Pull — bring down commits someone else pushed (your P2 partner, your P3 teammates) and merge them into your own work:

```
git pull
```

Run `git pull` *before* you start a work session whenever you're not working completely alone, and before you push, if any time has passed since your last pull. Pushing when your local copy is behind `origin` fails on purpose — git is refusing to silently throw away commits you haven't seen yet.

Branches — for P2 and P3, not really for P1

A branch is a separate line of commits that doesn't affect `main` until you merge it back in. The point: try something without risking the working version everyone else is looking at.

```
git branch                      # list branches, * marks the current one
git switch -c try-heap-board     # create AND switch to a new branch
git switch main                  # switch back
git merge try-heap-board         # from main, bring those commits in
```

For a solo project (P1), you can ignore branches entirely and just commit to `main`. For P2 (pairs) and P3 (teams), a common pattern is: each person branches for their own feature, pushes the branch, and merges into `main` once it works — so a half-finished experiment never breaks the copy your partner is building on.

A merge conflict, on purpose

This will happen to you for real at some point, usually at a bad time, so causing one deliberately now — the same way Lab 0 has you cause a sanitizer report on purpose — means the first time it happens for real, you already know what you're looking at.

1. Make sure `main` is committed and clean (`git status` shows nothing to commit).
2. Edit the **same line** of a file two different ways, in two branches:

```
git switch -c branch-a
# edit README.md, change the first line to "Hello from A"
git add README.md && git commit -m "Branch A edit"

git switch main
git switch -c branch-b
# edit README.md, change that SAME first line to "Hello from B"
git add README.md && git commit -m "Branch B edit"

git switch main
git merge branch-a      # succeeds, no conflict yet
git merge branch-b      # CONFLICT — same line, two different edits
```

3. Git stops and marks the file. Open it — you'll see something like:

```
<<<<<<< HEAD
Hello from A
=======
Hello from B
>>>>>>> branch-b
```

Everything between `<<<<<<< HEAD` and `=======` is what's already on your current branch. Everything between `=======` and `>>>>>>> branch-b` is what the branch you're merging in wants instead. Git cannot guess which one you want — that's the whole reason it stopped.

4. **Resolve it by hand**: delete the `<<<<<<<`, `=======`, `>>>>>>>` marker lines and whichever side you don't want, leaving the file exactly as it should read afterward. (You can also keep both, rewritten together, if that's what actually makes sense.)
5. Stage and commit the resolution — a merge conflict is resolved by a normal commit, there is no special "conflict commit" command:

```
git add README.md
git commit -m "Merge branch-b, keep the B greeting"
```

6. Clean up the practice branches, you don't need them:

```
git branch -d branch-a branch-b
```

The two mistakes everyone makes the first real time: panicking and deleting the file to start over (you'll lose both sides' work — just read the markers), and forgetting step 4 — committing the file *with the `<<<<<<<` markers still in it*, which silently corrupts the file instead of fixing anything. `git status` will tell you a merge is unresolved; don't commit until it doesn't.

Common mistakes

- **Committing generated files** (`build/`, compiled binaries). This repo's `.gitignore` already excludes `build/` — don't fight it or commit anyhow.
- **Forgetting to `git add` a new file** before committing — `git status` would have shown it as untracked; that's why you check first.
- **Writing "fixed stuff" as a commit message.** Six months from now (or six hours, during an oral code defense) that tells you nothing.
- **Committing a secret.** Never commit an API key, including a DeepSeek key — see `docs/SETUP_WSL2.md` step 7. If you ever do, treat the key as compromised (regenerate it) rather than assuming deleting the file later removes it from history — it doesn't; the old commit still has it.
- **Working for hours with no commits.** If your editor crashed right now, how much would you lose? That's roughly how often you should be committing.

Quick reference

Command	What it does
`git status`	What changed, what's staged — run this constantly
`git add <file>` / `git add -A`	Stage a file / stage everything
`git commit -m "message"`	Snapshot what's staged
`git log --oneline`	History, newest first
`git clone <url>`	Get a copy of a Gitea repo
`git push`	Send your commits to Gitea — this is submission
`git pull`	Bring down teammates' commits
`git branch`	List branches
`git switch -c <name>`	Create and switch to a new branch
`git switch <name>`	Switch to an existing branch
`git merge <name>`	Bring another branch's commits into the current one
`git diff`	Show unstaged changes, line by line